mndoc::mndoc 0.17.0

DETLEF GROTH, UNIVERSITY OF POTSDAM, GERMANY
2026-10-31

NAME

mndoc::mndoc - Tcl package and command line application to convert Markdown files to HTML files and to extract and format embedded programming documentation from source code files written in Markdown or doctools format. Local images and stylesheets can be embedded into the output document as base64 data. The mndoc tool supports standard Markdown with a few extensions which are explained below.

USE CASES

- simple install with small file size < 150kb
- convert Markdown files to HTML
- extract in source code embedded Markdown documentation
- embed local images into HTML files
- use provided extensions for better display experiences

TABLE OF CONTENTS

- SYNOPSIS
- DESCRIPTION
- COMMAND
- EXAMPLE
- FORMATTING
  - Title, Author, Date
  - Styling with CSS

    - Headers
    - Lists
    - Hyperlinks
    - Indentations
    - Font styles
    - Images
    - Code Blocks
    - Equations
    - Includes
- INSTALLATION
- SEE ALSO
- CHANGES
- TODO
- AUTHOR
- LICENSE AND COPYRIGHT

SYNOPSIS

Usage as package:

```
package require mndoc::mndoc
mndoc::mndoc inputfile outputfile ?--css file1.css,file2.css? \
   ?--header header.html? ?--footer footer.html? ?--base64 true? ?--bodyonly false?\
   ?--javascript highlightjs|file1.js,file2.js? ?--mathjax true? ?--refresh 10?
```

Usage as command line application for conversion of Markdown to HTML:

```
mndoc inputfile.md outputfile.html ?--css file.css,file2.css --header header.html \
  --footer footer.html --javascript highlightjs|filename1,filename2  --mathjax true \
  --refresh 10 --base64 true --bodyonly false?
```

Usage as command line application for conversion of Markdown to PDF:

```
mndoc inputfile.md outputfile.pdf ?options?
```

This first writes the matching `outputfile.html` and then prints it to PDF with headless chrome/chromium, or with weasyprint if no browser is installed. Without either of those tools

the application refuses to run, because then only HTML documents can be produced. The paper size follows the `LC_PAPER` locale (A4 by default, US Letter for locales such as `en_US`) and can be overridden with the environment variable `PAPER`, for instance `PAPER=Letter`.

The only exception is an input file which already is the HTML document behind that PDF, that is when infile and outfile share their base name, for example `mndoc chapter.html chapter.pdf`. The input file must not be overwritten, so no separate HTML file is written then, the PDF is printed from a temporary inlined copy of the input instead.

The options may be placed before, between or after the two file arguments, each option directly followed by its value, for example `mndoc --css file.css --bodyonly false inputfile.md outputfile.html`.

Usage as command line application for extraction of Markdown comments prefixed with `#'`:

```
mndoc inputcodefile outputfile.md ?options?
```

Usage as command line application to check for and install a newer release:

```
mndoc --upgrade
```

The running version is compared with the release tag GitHub serves for `releases/latest` (an HTTP redirect to `releases/tag/vX.Y.Z`). If a newer release exists, the official `install-mndoc.sh` installer is downloaded and run again, which refreshes the standalone binary in `~/.local/bin`. The check needs `curl` or `wget` and network access, the installation runs without asking questions, and a source checkout of the mndoc repository is not touched - use `git pull` there.

DESCRIPTION

mndoc::mndoc converts Markdown files to HTML files and extracts embedded Markdown or doctools documentation from source code files. The documentation inside the source code must be prefixed with the `#'` character sequence. The file extension of the output file determines the output format. File extensions can bei either `.md` for Markdown output, `.man` for doctools output or `.html` for html output. The latter requires the tcllib Markdown or the doctools extensions to be installed. If the file extension of the inputfile is .md and file extension of the output files is .html there will be simply a conversion from a Markdown to a HTML file. The command line application additionally accepts a .pdf output file: it writes the matching HTML file and prints it to PDF, see the SYNOPSIS section above for the details and the requirements on chrome/chromium or weasyprint.

The file `mndoc.tcl` can be as well directly used as a console application. An explanation on how to do this, is given in the section Installation.

COMMAND

mndoc::mndoc infile outfile ?--css file.css --header header.html --footer footer.html --mathjax true --refresh 10?

Extracts the documentation in Markdown format from infile and writes the documentation to outfile either in Markdown, Doctools or HTML format.

- infile - file with embedded markdown documentation
- outfile - name of output file extension
- --base64 false|true should local images and CSS files be embedded as base64 data, default: true
- --css cssfile if outfile is an HTML file use the given cssfile, one of the bundled stylesheets 'mndoc.css', 'tmdoc.css' or the flat, greyish book layout 'book.css', as well as any other local or remote stylesheet. Local stylesheets are resolved relative to the document being converted rather than the current working directory: with --base64 true (default) they are inlined as base64 data, with --base64 false a stylesheet bundled with mndoc is copied next to the output file so that the relative link stays valid
- --footer footer.html if outfile is an HTML file add this footer before the closing body tag
- --header header.html if outfile is an HTML file add this header after the opening body tag
- --javascript highlightjs|filename1,filename2 if outfile is an HTML file embeds either the highlight.js Javascript highlighter or the given local javascript filename(s)
- --mathjax false|true should there be the MathJax library included, default: false
- --refresh 0|10 should there be the autorefresh header included only values above 9 are considered, default: 0
- --bodyonly false|true should only the part within the body tags be saved to the new file, default: false

If the file extension of the outfile is either html or htm a HTML file is created. If the output file has other file extension the documentation after #' comments is simply extracted and stored in the given outfile, -mode flag (one of -html, -md, -pandoc) is not given, the output format is taken from the file extension of the output file, either .html for HTML or .md for Markdown format. This deduction from the filetype can be overwritten giving either `-html` or `-md` as command line flags. If as mode `-pandoc` is given, the Markdown markup code as well contains the YAML header. If infile has the extension .md (Markdown) or -man (Doctools) than conversion to html will be performed, the outfile file extension In this case must be .html. If output is html a --css flag can be given to use the given stylesheet file instead of the default style sheet embedded within the mndoc code. As well since version 0.8.0 a --header and --footer option is available to add HTML code at the beginning and at the end of the document.

EXAMPLE

```
package require mndoc::mndoc
mndoc::mndoc mndoc.tcl mndoc.html                  ## simple HTML page
mndoc::mndoc mndoc.tcl mndoc.md                    ## just output a Markdown page
mndoc::mndoc mndoc.tcl mndoc.html --refresh 20     ## reload HTML page every
                                                   ## twenty seconds
mndoc::mndoc mndoc.tcl mndoc.html --mathjax true   ## parse inline equations
                                                   ## using mathjax library
mndoc::mndoc sample.html sample-out.html           ## inline images and stylesheets
                                                   ## into sample-out.html
mndoc::mndoc header.md header.html --bodyonly true ## no HTML header, no HTML footer
```

FORMATTING

For a complete list of Markdown formatting commands consult the basic Markdown syntax at https://daringfireball.net. Here just the most basic essentials to create documentation are described. Please note, that formatting blocks in Markdown are separated by an empty line, and empty line in this documenting mode is a line prefixed with the `#'` and nothing thereafter.

Title, Author and Date

Title, author and date can be set at the beginning of the documentation in a so called YAML header. This header will be as well used by the document converter pandoc to handle various options for later processing if you extract not HTML but Markdown code from your documentation.

A YAML header starts and ends with three hyphens. Here is the YAML header of this document:

```
#' ---
#' title: mndoc - Markdown extractor and formatter
#' author: Dr. Detlef Groth, Schwielowsee, Germany
#' date: 2025-01-04
#' ---
```

Those five lines produce the three lines on top of this document. You can extend the header if you would like to process your document after extracting the Markdown with other tools, for instance with Pandoc.

The date value is normalized to `YYYY-MM-DD` if it is an ISO date such as `2026-10-28` or `2026-10-28T07:33:23`; time and timezone information are dropped. Unix timestamps in seconds or milliseconds are converted as well. All other values, for example `28.10.2026`, are kept exactly as written. This avoids stopping the conversion because of an unusual date format.

CSS Styles

You can as well specify an other style sheet, than the default by adding the following style information:

```
#' ---
#' title: mndoc - Markdown extractor and formatter
#' author: Dr. Detlef Groth, Schwielowsee, Germany
#' date: 2024-11-21
#' css: tufte.css
#' ---
```

If you like to change the default font it is recommended to place this informaton either in the css file, or if you like you can add as well a style section to the YAML either like in this example:

```
#' ---
#' title: mndoc::mndoc 0.17.0
#' author: Detlef Groth, University of Potsdam, Germany
#' date: 2026-10-31
#' css: mndoc.css
#' style: |
#'   @import url('https://fonts.bunny.net/css?family=Andika&display=swap');
#'   @import url('https://fonts.bunny.net/css?family=Ubuntu+Mono&display=swap');
#'   body { font-family: Andika, sans-serif ; }
#'   pre, code { font-family: "Ubuntu Mono", monospaced ; }
#' ---
```

The use of the fonts provided by https://fonts.bunny.net is recommended if you are living in a member state of the European Union.

Headers

Headers are prefixed with the hash symbol, single hash stands for level 1 heading, double hashes for level 2 heading, etc. Please note, that the embedded style sheet centers level 1 and level 3 headers, there are intended to be used for the page title (h1), author (h3) and date information (h3) on top of the page.

```
#'  ## <a name="sectionname">Section title</a>
#'
#'  Some free text that follows after the required empty
#'  line above ...
```

This produces a level 2 header. Please note, if you have a section name `synopsis` the code fragments thereafter will be highlighted differently than the other code fragments. You should only use level 2 and 3 headers for the documentation. Level 1 header are reserved for the title.

Lists

Lists can be given either using hyphens or stars at the beginning of a line.

```
#' - item 1
#' - item 2
#' - item 3
```

Here the output:

- item 1
- item 2
- item 3

A special list on top of the help page could be the table of contents list. Here is an example:

```
#' ## Table of Contents
#'
#' - [Synopsis](#synopsis)
#' - [Description](#description)
#' - [Command](#command)
#' - [Example](#example)
#' - [Authors](#author)
```

This will produce in HTML mode a clickable hyperlink list. You should however create the name targets using html code like so:

```
 <a name='synopsis'>Synopsis2</a>
```

Another type of lists are TODO lists. mndoc has limited support for it. They are simple unnested lists starting at the beginning of a line with a minus sign folled by either `[ ]` for a TODO item or a `[x]` indicator for a done item. Here an example:

```
- [ ] Todo item 1
- [ ] TODO item 2
- [x] DONE item 1
- [x] another item which was done already
```

Here the output:

- ☐ Todo item 1
- ☐ TODO item 2
- ☑ DONE item 1
- ☑ another item which was done already

Hyperlinks

Hyperlinks are written with the following markup code:

```
[Link text](URL)
```

Let's link to the Tcler's Wiki:

```
[Tcler's Wiki](https://wiki.tcl-lang.org/)
```

produces: Tcler's Wiki

Indentations

Indentations are achieved using the greater sign:

```
#' Some text before
#'
#' > this will be indented
#'
#' This will be not indented again
```

Here the output:

Some text before

> this will be indented

This will be not indented again

Also lists can be indented:

```
> - item 1
  - item 2
  - item 3
```

produces:

> - item 1
> - item 2
> - item 3

Font Styles

Italic font style can be requested by using single stars or underlines at the beginning and at the end of the text. Bold is achieved by dublicating those symbols: Monospace font appears within backticks. Here an example:

```
#' > I am _italic_ and I am __bold__! But I am programming code: `ls -l`
```

> I am italic and I am **bold**! But I am programming code: `ls -l`

Images

If you insist on images in your documentation, images can be embedded in Markdown with a syntax close to links. The links here however start with an exclamation mark:

```
#' ![image caption](filename.png)
```

Image attributes like width can be given after the image code like this (ignore the space after the closing parenthesis):

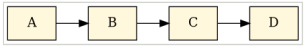
```
![dot image](../examples/dot.png){width="150px"}
```

A unitless value means pixels as well, so width="150" and width="150px" are the same suggestion, and a percentage like width="50%" is accepted too. mndoc restates the size as an

inline style, because the width and height attributes on their own are only presentational hints: PDF renderers such as weasyprint ignore them completely and any stylesheet rule outranks them. The author's size is thus taken as the default, while a stylesheet may still apply a ceiling of its own, for instance max-width: 100%.

An image that is the only content of its paragraph or list item, one that stands alone, is additionally marked with a data-figure attribute in the output. CSS cannot make that distinction itself: text nodes cannot be selected, so an image sitting mid-sentence looks exactly like a lone one to a selector. The bundled book.css uses the marker to set such a plate on its own line; the marker is inert in any other stylesheet.

Here the output:



The source code of mndoc.tcl is a good example for usage of this source code annotation tool. Don't overuse the possibilities of Markdown, sometimes less is more. Write clear and concise, don't use fancy visual effects.

Code blocks

Code blocks can be started using either three or more spaces after the #' sequence or by embracing the code block with triple backticks on top and on bottom. Here an example:

```
#' ```
#' puts "Hello World!"
#' ```
```

Here the output:

```
puts "Hello World!"
```

Since version 0.8.0 mndoc as well included the inclusion of Javascript files or libraries like the library highlight.js. Just use the command line or mndoc function argument `--javascript highlightjs` and you get syntax highlighting for code blocks. Here an example:

```
#' ```{r}
#' test <- function () {
#'   print("testig2")
#' test();
#' ```
```

Output:

```
test <- function () {
  print("testig2")
test();
```

Equations

Since version 0.9.0 as well LaTeX equations can be embedded into Markdown documents and are rendered using the MathJax library. Just include either inline equations using parenthesis protected by two backslashes or block equations embedded within brackets protected by two backslashes or within two dollar symbols in your Markdown code and use the option `--mathjax true` during document conversion. Here an example for inline equations:

```
The  famous  Einstein  equation  \\( E = mc^2 \\) is  probably  the most known
equation world wide.
```

And here the output (will not work on http://htmlpreview.github.io/):

The famous Einstein equation \(E = mc^2 \) is probably the most know equation world wide.

Block equations should be usually aligned left, like in the following examples:

```
<div style="display: flex;">

$$ \sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6} \tag{1} $$

</div>
```

And here the output (please note that this does not work in Github Preview Mode):

$$ \sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6} \tag{1} $$

Includes

mndoc in contrast to standard markdown as well support includes. Using the

```
#' #include "filename.md"
```

syntax it is possible to include other markdown files. This might be useful for instance to include the same header or a footer in a set of related files.

INSTALLATION

The mndoc::mndoc package can be installed either as command line application or as a Tcl module. It requires the markdown, cmdline, yaml and textutil packages from tcllib to be installed.

Installation as command line application is easiest by executing the following shell one liner:

```
/bin/bash -c \
 "$(curl -fsSL https://github.com/mittelmark/mndoc/releases/latest/download/install-mndoc.sh)"
```

Alternatively it can be installed by downloading the file mndoc-0.1X.X.bin from the latest release, which contains the main script file and all required libraries, to your local machine. The X stands for the current release number. Rename this file to mndoc, make it executable and coy it to a folder belonging to your PATH variable.

Installation as command line application can be as well done by copying the `mndoc.tcl` as `mndoc` to a directory which is in your executable path. You should make this file executable using `chmod`.

Installation as Tcl package by copying the mndoc folder to a folder which is in your library path for Tcl. Alternatively you can install it as Tcl modul by copying it in your module path as `mndoc-0.1X.X.tm` for instance. See the tm manual page

SEE ALSO

- tmdoc for an approach to perform literate programming using Tcl
- tcllib for the original mkdoc package and the Markdown as well as the textutil packages
- pandoc - am universal document converter
- Ruff! Ruff! documentation generator for Tcl using Markdown syntax as well

CHANGES

- 2026-11-02 Release 0.17.0
  - flexible command line option order: options may be placed before, between or after the two file arguments, each directly followed by its value
  - new .pdf output: an OUTFILE with the .pdf extension first writes the matching HTML file and prints it to PDF with headless chrome/chromium or weasyprint (both engines

render MathJax equations and highlight.js code, WeasyPrint alone cannot); without either tool mndoc refuses, as then only HTML documents can be produced
    - shared print fit in the stylesheets (@media print) and the paper size now follows the locale: A4 by default, US Letter for locales such as en_US, overridable with PAPER=...
    - mndoc --upgrade checks GitHub for a newer release (the release tag behind the releases/latest redirect) and, if one exists, runs the install-mndoc.sh installer again to refresh the standalone binary in ~/.local/bin
    - -h alias for --help; help page repositioned with Markdown to HTML conversion first
- 2026-10-31 Release 0.16.0
    - new book.css stylesheet: a flat, greyish layout in the style of a traditional book, self-contained without remote font imports
    - images that are the only content of their paragraph or list item are marked with data-figure, which book.css uses to tell a figure from an inline ornament; CSS alone cannot tell them apart, because text nodes are invisible to selectors
    - explicit image sizes are honored: width="800" is restated as an inline style and the width/height attributes are normalized to the unitless integers the HTML spec requires, so renderers like weasyprint no longer drop the author's size; while no size is given, stylesheet defaults such as the book text-height cap apply
    - reading measure widened from 800px to 1000px; mndoc.css is now centred with a 1000px maximum width
    - fix --css embedding to resolve relative to the document rather than the current working directory
    - declare the output charset so UTF-8 text renders correctly
- 2026-10-27 Release 0.15.5
    - fix YAML date handling for non-ISO date formats; unknown dates are kept as written instead of aborting the conversion
- 2026-01-28 Release 0.15.4
    - fixing fixing LICENSE issues stating mixes of BSD/MIT, LICENSE should be BSD3
    - fixing more issues with pipes in text
- 2026-01-20 Release 0.15.3
    - fixing issues with pipes in text, leading to tables, can be avoided by adding backticks around pipes
- 2025-12-12 Release 0.15.2
    - also files with extension .Rmd, .rmd, .Pmd, .pmd, .Tmd and .tmd are taken as Markdown input
    - support for br-tags in yaml title section

- 2025-12-01 Release 0.15.1
  - fix for non-existing stylesheet files like mndoc.css or tmdoc.css taking them from the package folder
- 2025-11-20 Release 0.15.0
  - support for """#' and /#' to start Markdown blocks and """ */ to end Markdown blocks to document for example the Julia language
- 2025-10-30 Release 0.14.2
  - mathjax inline equations with $ equation $ to avoid backslash issues
- 2025-10-26 Release 0.14.1
  - file application cache file right fix for multiple users on the same machine try to run mndoc
- 2025-10-23 Release 0.14.0
  - adding support for inlining local images and stylesheets into existing HTML files
  - adding option --bodyonly to omit HTML header and footer as well as body tag
  - support for style section in YAML header for instance to install and use Bunny fonts
  - support for simple todo lists
  - support for image attributes like width
- 2025-10-16 Release 0.13.0
  - renamed to mndoc with version 0.13.0 to avoid name collisions with mkdoc package in tcllib
- 2025-01-26 Release 0.11.3
  - fixing wrong command line argument crash
  - fixing uneven length option list
- 2025-01-18 Release 0.11.2
  - fixing inline multiple images on the same line
- 2025-01-04 Release 0.11.1
  - fixing outfile ending with Tmd, Rmd etc seen as HTML files
- 2025-01-04 Release 0.11.0
  - Tcl 9 support
- 2024-12-24 Release 0.10.2
  - amp-amp fix for source code blocks
- 2024-11-28 Release 0.10.1
  - minor documentation fix
- 2024-11-28 Release 0.10.0
  - support for refresh option to autorefresh a HTML page
  - removed run support, use pantcl instead

    - fixing issues with greater, lower and quote signs in code fragments
    - removing inlining external javascript files into HTML output
    - adding --base64 option to inline local images and css files
- 2024-11-16 Release 0.9.0
    - support for mathjax
- 2023-11-17 Release 0.8.0
    - removed hook package, sorry do not understand what it is doing and what is the benefit and I could not extend my code with this
    - adding --header and --footer options
    - adding --javascript option, single oder multiple files
    - extending --css option, single or multiple files
    - support for syntax highlighting using highlight.js Javascript
    - fixing issues with triple backtick codes, by fixing markdown package (issue is done on tcllib)
    - adding example file in examples to show syntax highlighting
    - adding Makefile to build standalone application using tpack (80kb)
- 2023-09-07 Release 0.7.1 - image tag fix
- 2022-04-01 Release 0.7.0
    - removing features to simplify the code, so removed plugin support, underline placeholder and sorting facilities to reduce code size
    - creating tcllib compatible manual page
    - aku changes and fixes to include mndoc into tcllib's infrastructure
    - splitting of command line app to the apps folder
    - adding hook package requirement (benefit?)
    - changing license to BSD
- 2022-02-11 Release 0.6.0
    - parsing yaml header
    - workaround for images
    - making standalone using tpack.tcl mndoc-0.6.bin
    - terminal help update and cleanup
    - moved to Github in Wiki
    - code cleanup
- 2020-12-30 Release 0.5 (rox2md @section support with preformatted, emph and strong/bold)
- 2020-11-11 command line option --run with seconds
- 2020-11-10 Release 0.4

- 2020-11-09 argument --run support
- 2020-09-01 Roxygen2 plugin
- 2020-02-27 support for __DATE__, __PKGNAME__, __PKGVERSION__ macros in Tcl code based on package provide line
- 2020-02-26 Release 0.3 to fossil
- 2020-02-26 Making standalone file using pkgDeps and mk_tm
- 2020-02-26 Adding stylesheet option --css
- 2020-02-26 Adding files pandoc.css and dgw.css
- 2020-02-02 Adding include syntax
- 2020-01-05 Documentation fixes and version information
- 2019-12-06 Partial R-Roxygen/Markdown support
- 2019-11-28 Release 0.2 to fossil
- 2019-11-28 Kit version
- 2019-11-27 Documentation fixes
- 2019-11-22 Adding direct conversion from Markdown files to HTML files.
- 2019-11-19 Release 0.1

TODO

- ☑ font embedding using https://european-alternatives.eu/de/produkt/bunny-fonts currently Ubuntu Mono and Andika are used from there within the default stylesheet (done)
- ☐ dtplite support ?
- ☐ inline online images and online stylesheets?

AUTHOR(s)

The mndoc::mndoc package was written by Detlef Groth, University of Potdam, Germany.

LICENSE AND COPYRIGHT

BSD 3-Clause License

Copyright (c) 2019-2026, Detlef Groth

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.